ELO320 - Estructuras de Datos y Algoritmos
Tipos de Datos Abstractos
Árbol binario de Búsqueda

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2022

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

ABB: Árbol binario de búsqueda

Un árbol binario de búsqueda es una implementación del árbol binario que permite guardar y buscar eficientemente de datos debido a su estructura.

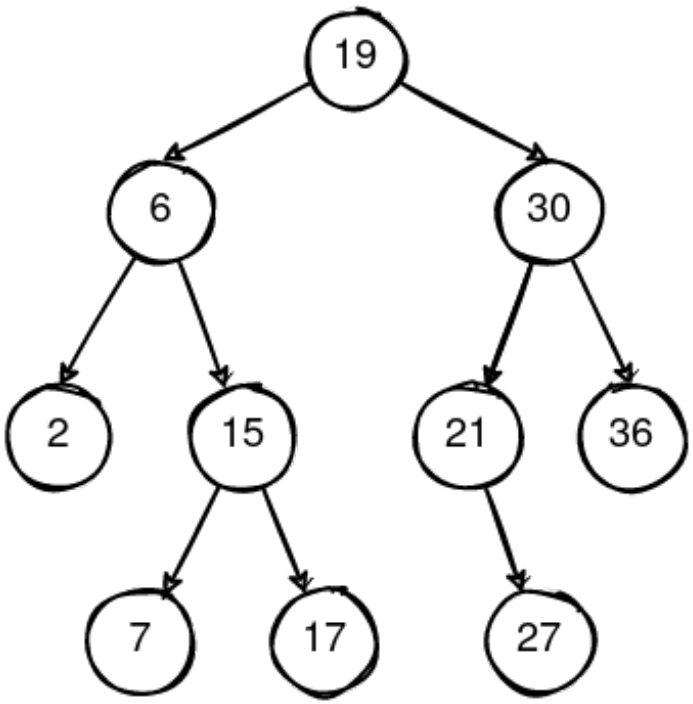


ABB: Árbol binario de búsqueda

En un árbol binario de búsqueda, los hijos izquierdos guardan información menor (o igual), y los hijos derechos guardan información mayor, con respecto a la de su padre.

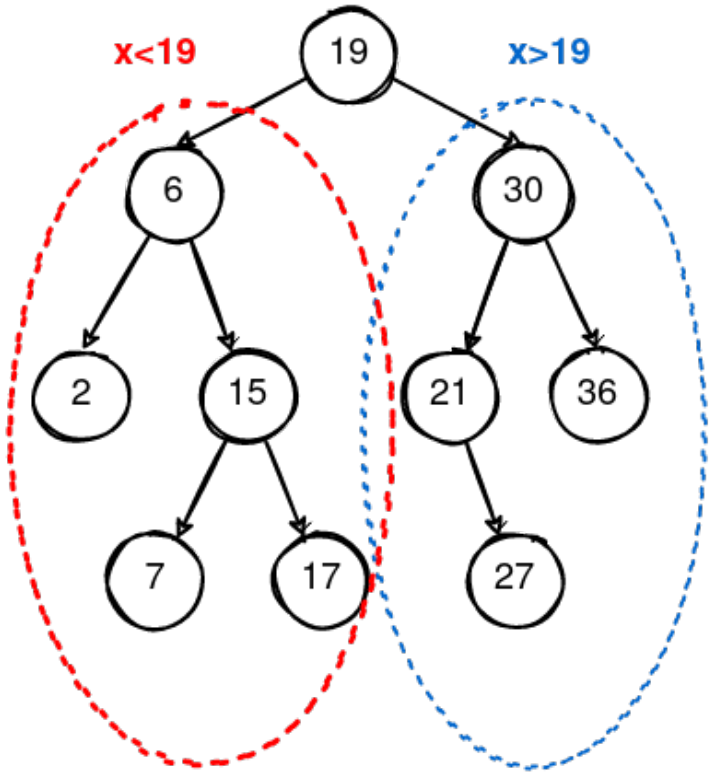
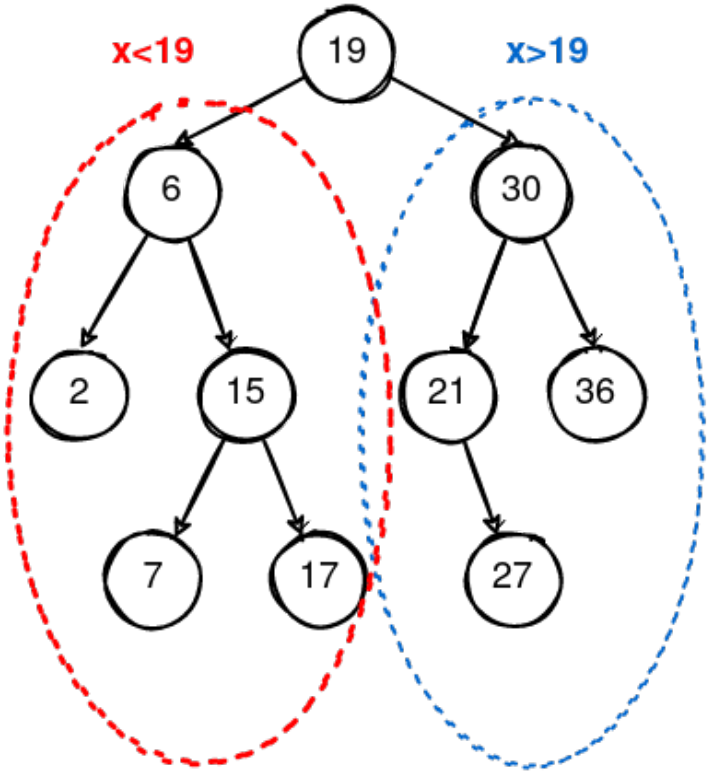


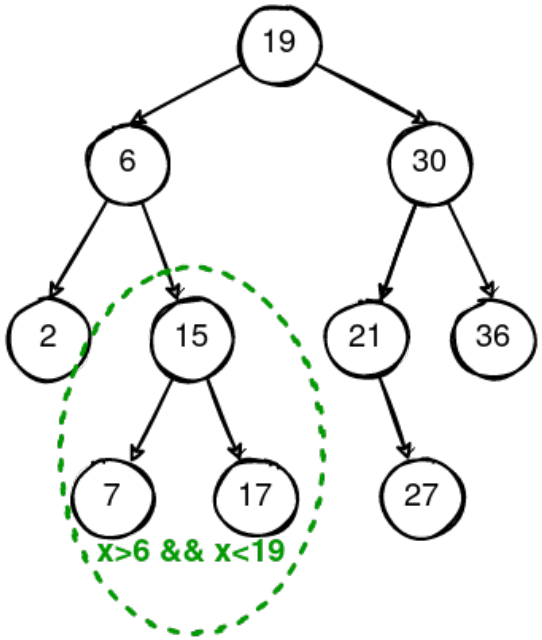
ABB: Árbol binario de búsqueda



A partir de esta regla de modelamiento se definen fácilmente las operaciones de búsqueda, inserción, y eliminación de elementos. Además de que los recorridos pre-orden, in-orden y post-orden sean efectivos para determinadas labores.

ABB: Árbol binario de búsqueda

Por ejemplo, es fácil determinar el lugar y posición de los valores en un intervalo.



- **¿Cómo determinan el sucesor y predecesor de un padre?**
  Sucesor: buscar en la derecha, todo hacia la izquierda.
  Antecesor: buscar en la izquierda, todo hacia la derecha.
- **¿Cuál es su principal característica?**
  Siempre tendrán a lo más un hijo.

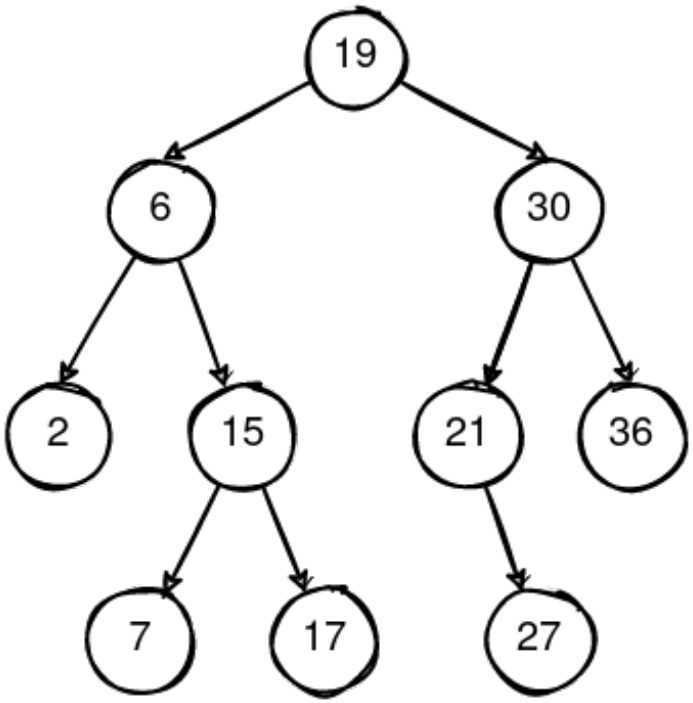
ABB: Operaciones

Las operaciones más comunes de los ABB son:

- `init()`: Inicializar el árbol (root == NULL).
- `size()`: Cantidad de elementos en el árbol.
- `x_preorden()`: Operación x en pre-orden.
- `x_inorden()`: Operación x en in-orden.
- `x_postorden()`: Operación x en post-orden.
- `search()`: Buscar un elemento en el árbol.
- `insert()`: Insertar un elemento en el árbol.
- `delete()`: Eliminar un elemento en el árbol.
- `clear()`: Limpiar el árbol.

ABB: Recorrer

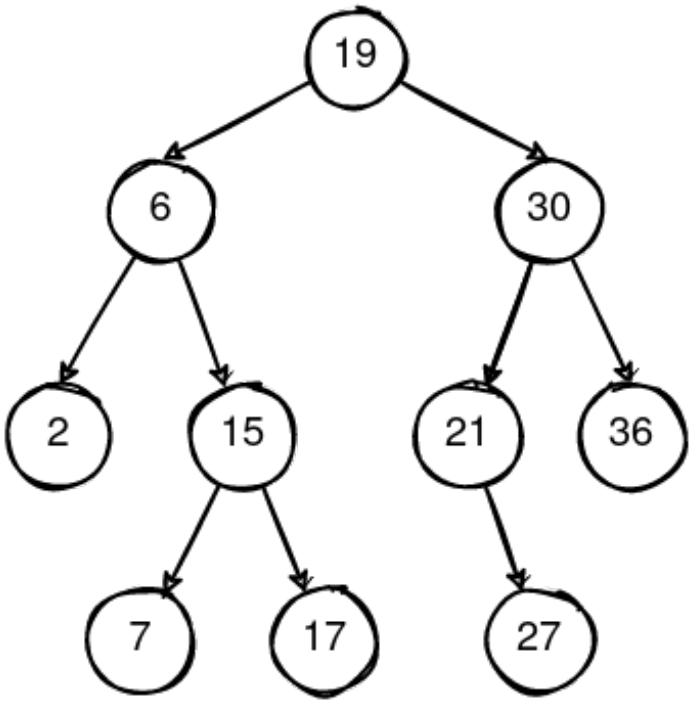
¿Como se imprimirían los datos del siguiente árbol con recorrido pre-orden, in-orden y post-orden?



- Pre-orden: 19 - 6 - 2 - 15 - 7 - 17 - 30 - 21 - 27 - 36
- In-orden: 2 - 6 - 7 - 15 - 17 - 19 - 21 - 27 - 30 - 36
- Post-orden: 2 - 7 - 17 - 15 - 6 - 27 - 21 - 36 - 30 - 19

ABB: Recorrer

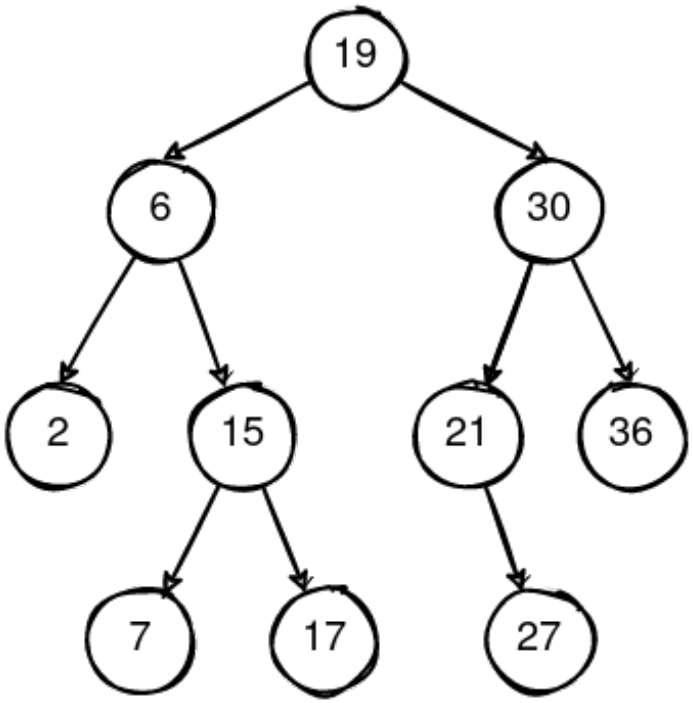
¿Como se imprimirían los datos del siguiente árbol con recorrido pre-orden, in-orden y post-orden?



- Pre-orden: 19 - 6 - 2 - 15 - 7 - 17 - 30 - 21 - 27 - 36
- In-orden: 2 - 6 - 7 - 15 - 17 - 19 - 21 - 27 - 30 - 36
- Post-orden: 2 - 7 - 17 - 15 - 6 - 27 - 21 - 36 - 30 - 19

ABB: Recorrer

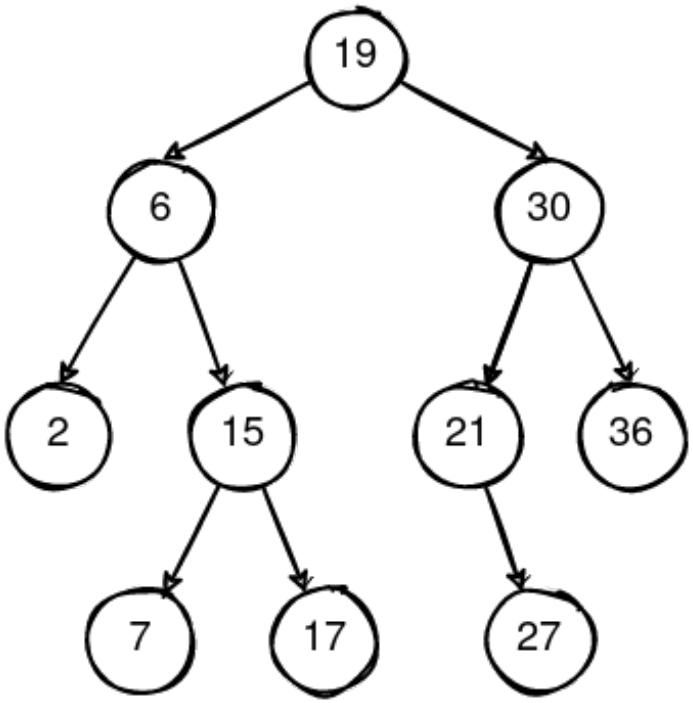
¿Como se imprimirían los datos del siguiente árbol con recorrido pre-orden, in-orden y post-orden?



- Pre-orden: 19 - 6 - 2 - 15 - 7 - 17 - 30 - 21 - 27 - 36
- In-orden: 2 - 6 - 7 - 15 - 17 - 19 - 21 - 27 - 30 - 36
- Post-orden: 2 - 7 - 17 - 15 - 6 - 27 - 21 - 36 - 30 - 19

ABB: Recorrer

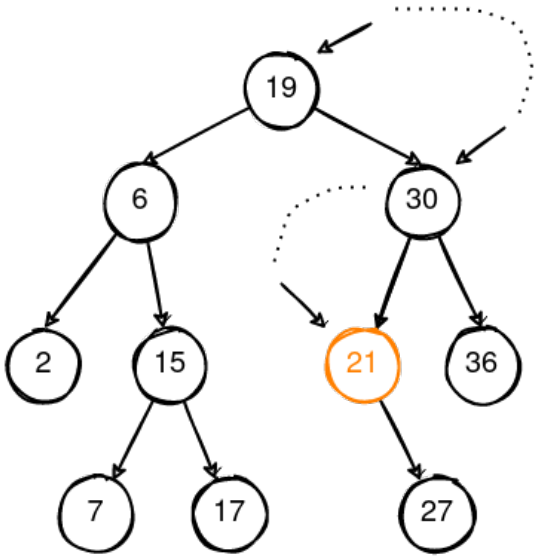
¿Como se imprimirían los datos del siguiente árbol con recorrido pre-orden, in-orden y post-orden?



- Pre-orden: 19 - 6 - 2 - 15 - 7 - 17 - 30 - 21 - 27 - 36
- In-orden: 2 - 6 - 7 - 15 - 17 - 19 - 21 - 27 - 30 - 36
- Post-orden: 2 - 7 - 17 - 15 - 6 - 27 - 21 - 36 - 30 - 19

ABB: Buscar

Comenzaremos con la función básica `search()`: Buscar un elemento en el árbol.



Se puede hacer de dos formas:

- Usando un puntero para recorrer el árbol.
- **Recursivamente**.

ABB: Buscar

Recursivamente

```
int * search(nodo_t * raiz, int value){

        if(raiz == NULL || raiz->dato == value) //{} omitidas
                return raiz;
        else if(value < raiz->dato)
                return search(raiz->hijo_izq, value);
        else if(value > raiz->dato) // else
                return search(raiz->hijo_der, value);

}
```

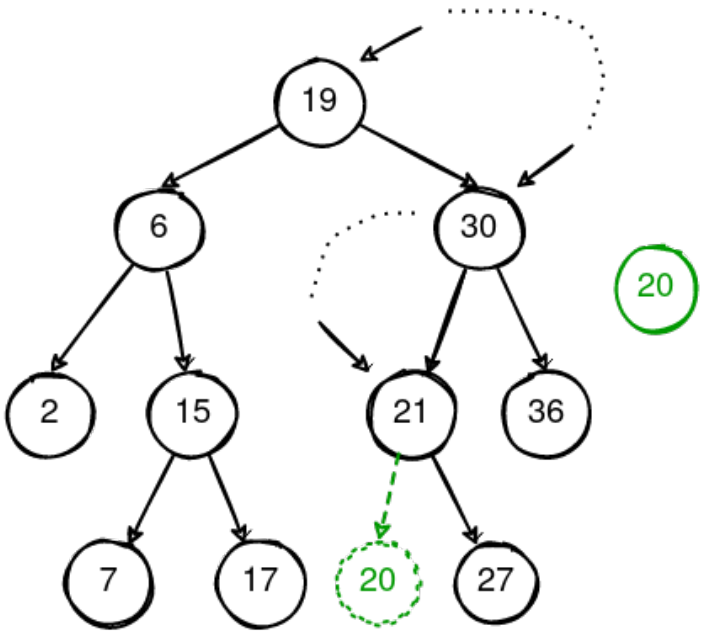
Usando Puntero

```
int * search(nodo_t * raiz, int value){

        nodo_t * rec = raiz;
        while(rec!=NULL){
                if(rec->dato == value){
                        return rec;
                }else if(value < rec->dato){
                        rec = rec->hijo_izq;
                }else if(value > rec->dato){
                        rec = rec->hijo_der;
                }
        return rec; // NULL si no lo encuentra o arbol vacío
}
```

ABB: Insertar

Para poder insertar debemos buscar primero el lugar donde debe realizarse la inserción. La idea es **mantener** el orden de nuestro ABB.



Nota: Si nuestro árbol no acepta elementos repetidos, al encontrar el valor se detiene.

ABB: Insertar

Recursivamente

```
void insert(nodo_t ** raiz, int value){

        if(*raiz == NULL){
                *raiz = malloc(sizeof(node_t));
                (*raiz)-> dato = value;
                (*raiz)->hijo_izq = NULL;
                (*raiz)->hijo_der = NULL;

        }else if(value < (*raiz)->dato){
                insert(&((*raiz)->hijo_izq), value);

        }else if(value > (*raiz)->dato){
                insert(&((*raiz)->hijo_der), value);
        }
}
```

ABB: Insertar

Cree el árbol binario que represente la siguiente secuencia de enteros:

$$10 - 5 - 7 - 14 - 12 - 18 - 15$$

Spoiler

ABB: Insertar

Cree el árbol binario que represente la siguiente secuencia de enteros:

$$10 - 5 - 7 - 14 - 12 - 18 - 15$$

Spoiler

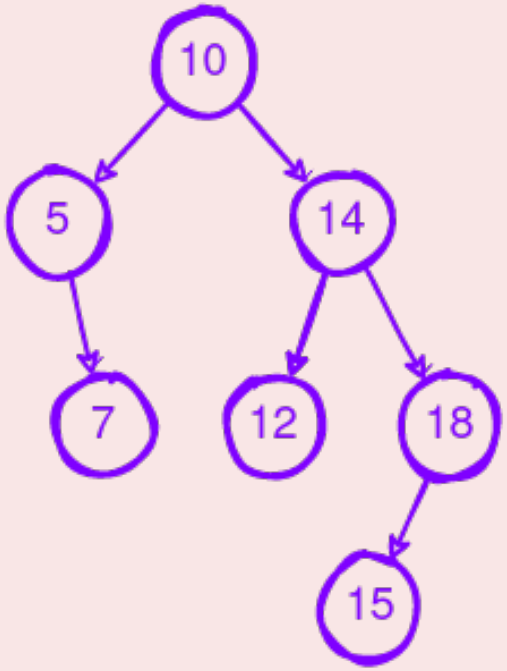


ABB: Eliminar

Al igual que en insertar, deberemos buscar el nodo a eliminar. Pero para poder eliminarlo, tendremos que considerar los siguientes casos:

- Nodo objetivo es una hoja (no tiene hijos).
- Nodo objetivo tiene un sólo hijo.
- Nodo objetivo tiene dos hijos.

ABB: Eliminar hoja.

Es el caso más sencillo, solo corresponde a liberar el nodo desde el puntero del padre.

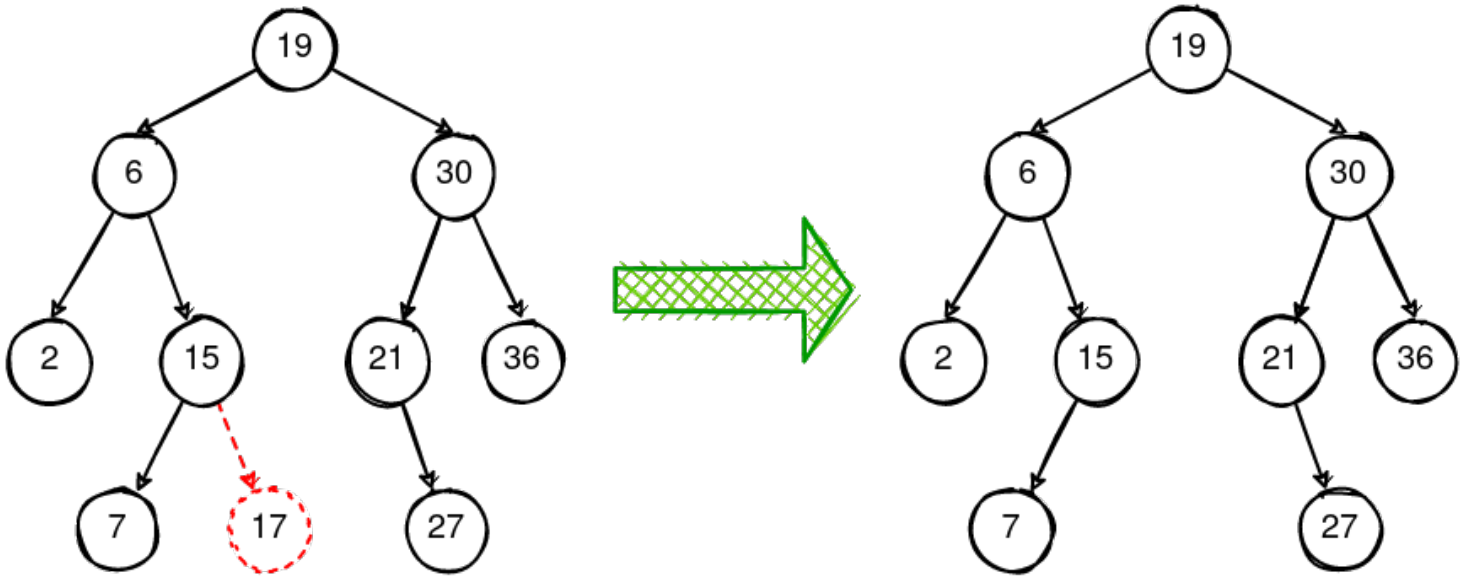


ABB: Eliminar nodo con 1 hijo.

Caso intermedio. acá se elimina el nodo encontrado, pero su posición es ocupada por su único hijo. Se eleva ese sub-árbol, un nivel.

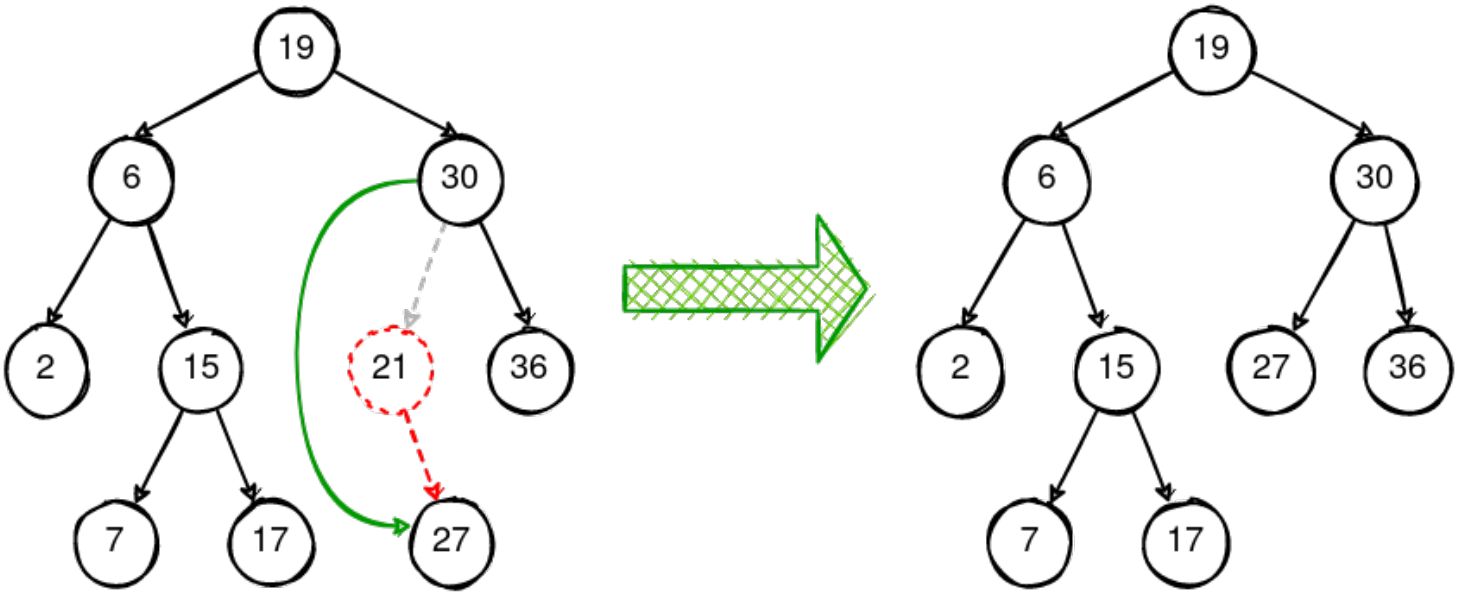


ABB: Eliminar nodo con 2 hijos.

Caso recursivo o iterativo. Cuenta de dos pasos:

- Reemplazar el nodo objetivo por una copia de su sucesor o antecesor (uds. eligen).
- Borrar el sucesor o antecesor original.

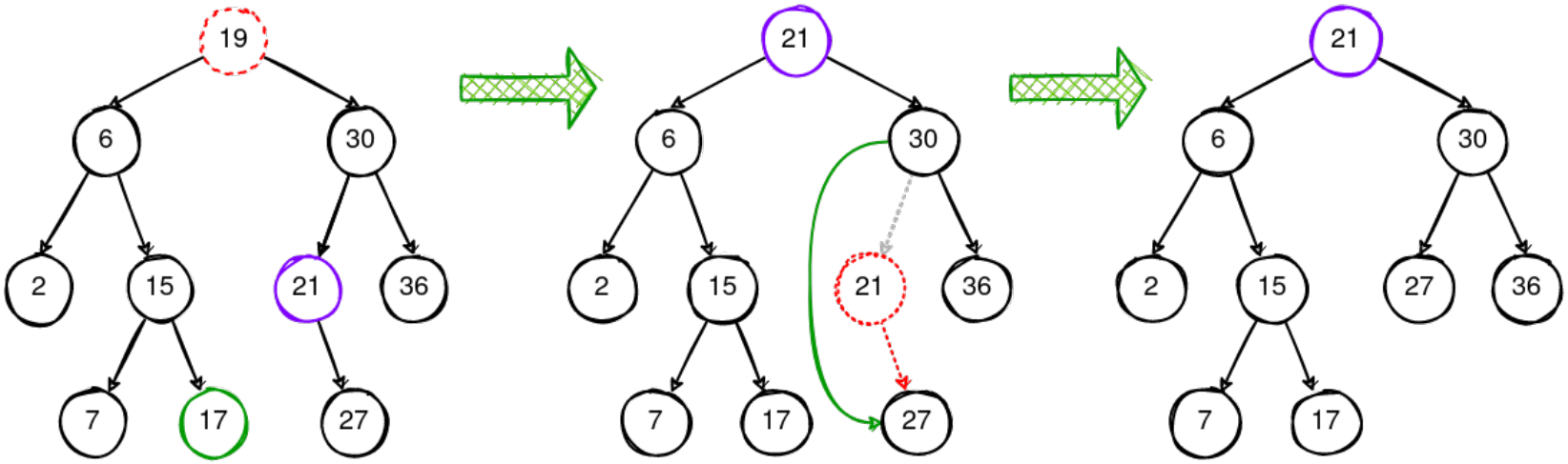


ABB: Insertar

Recursivamente

```
int delete(nodo_t ** raiz, int value){
        // ¿Cómo lo programan?
        // Necesitarán alguna función extra.
}
```

Ejercicios

- Para el siguiente conjunto $\{1, 4, 5, 10, 16, 17, 21\}$, dibuje árboles binarios de búsqueda con altura 2, 3, 4, 5, y 6.
- Escriba funciones recursivas para recorrer un árbol binario de búsqueda en pre-orden y post-orden.
- Asuma que tiene números entre 1 y 1000 en un árbol binario de búsqueda y quiere buscar el nodo 363. ¿Cuál de las siguientes secuencias de nodos NO podría ser una secuencia de nodos examinados en el árbol para llegar al nodo 363?
  1. 2, 252, 401, 398, 330, 344, 397, 363.
  2. 924, 220, 911, 244, 898, 258, 362, 363.
  3. 925, 202, 911, 240, 912, 245, 363.
  4. 2, 399, 387, 219, 266, 382, 381, 278, 363.
  5. 935, 278, 347, 621, 299, 392, 358, 363.